100 XP

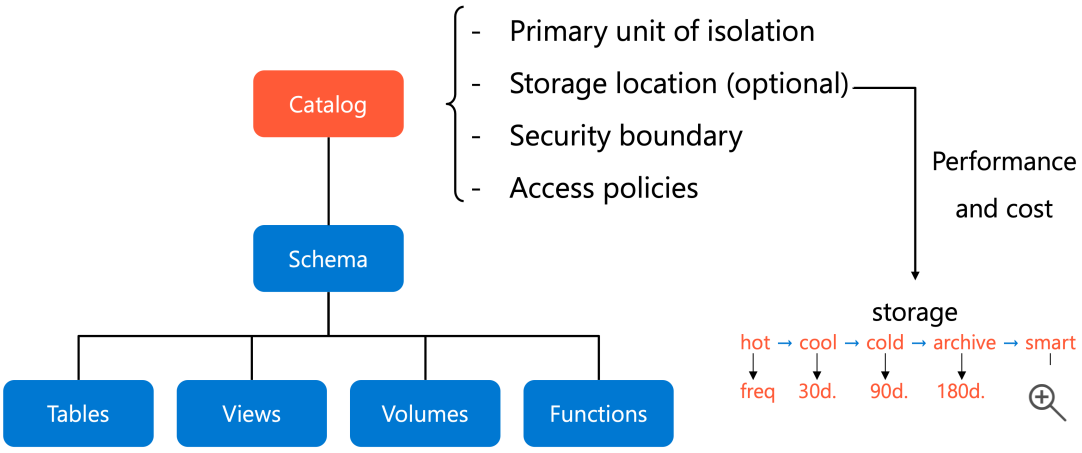
Create catalog

7 minutes

Your organization's data governance policies require clear separation between **development**, **testing**, and **production** environments. You need production data in dedicated storage, development work isolated from live systems, and sensitive customer information protected from operational data. **Unity Catalog's catalog structure** helps you meet these requirements while maintaining centralized governance and access control.

What is a catalog?

A **catalog** is the **top-level container** in Unity Catalog's **three-layer namespace**. Think of it as the foundation of your data organization, like building a house before decorating the rooms inside. Every data asset in Unity Catalog exists within this hierarchy: `catalog.schema.table`.



Catalogs serve as the primary unit of **data isolation** and organization. Each catalog can have its own **storage location**, **security boundaries**, and **access policies**. This physical separation ensures that your development experiments don't accidentally affect production data, and that sensitive information remains isolated from general-purpose datasets.

With only three layers available for organizing your data, understanding each layer's role becomes essential. Catalogs handle the highest level of separation, typically mapping to business domains, security requirements, or lifecycle stages. The schemas within those catalogs organize the interior, and volumes or tables store the actual data.

Organize data with catalogs

When you design your catalog structure, you're creating the foundation for **data governance** across your organization. Catalogs typically mirror organizational units or software development lifecycle scopes. Consider how your teams work and what data access boundaries matter most to your operations.

Environment-based isolation is one of the most common patterns. You might create separate catalogs for production, staging, and development environments. This separation prevents development queries from impacting production performance and ensures that experimental code never touches live customer data. With this pattern, your production catalog (`prod_catalog`) contains only validated, approved datasets, while your development catalog (`dev_catalog`) provides a safe space for testing and iteration.

Sensitivity-based isolation works well when compliance or privacy concerns drive your architecture. You might separate customer data from operational metrics, or isolate financial records from general business analytics. Each catalog can enforce different access policies, making it simple to grant broad access to public datasets while restricting sensitive information to authorized personnel.

Your catalog structure also affects **performance and costs**. Because each catalog can specify its own managed storage location, you can store frequently accessed production data in high-performance storage while archiving historical development data to more economical options. Regional compliance requirements become manageable too, since Unity Catalog metastores exist per region and catalogs inherit that regional isolation.

Create a catalog

Creating a catalog establishes the foundation for all data objects you'll add later. You must be a **metastore admin** or have the **CREATE CATALOG privilege** to create new catalogs. Once created,

the catalog includes two automatically generated schemas: **default** for your general use and **information_schema** for system metadata.

Using SQL, you can create a catalog with a simple command:

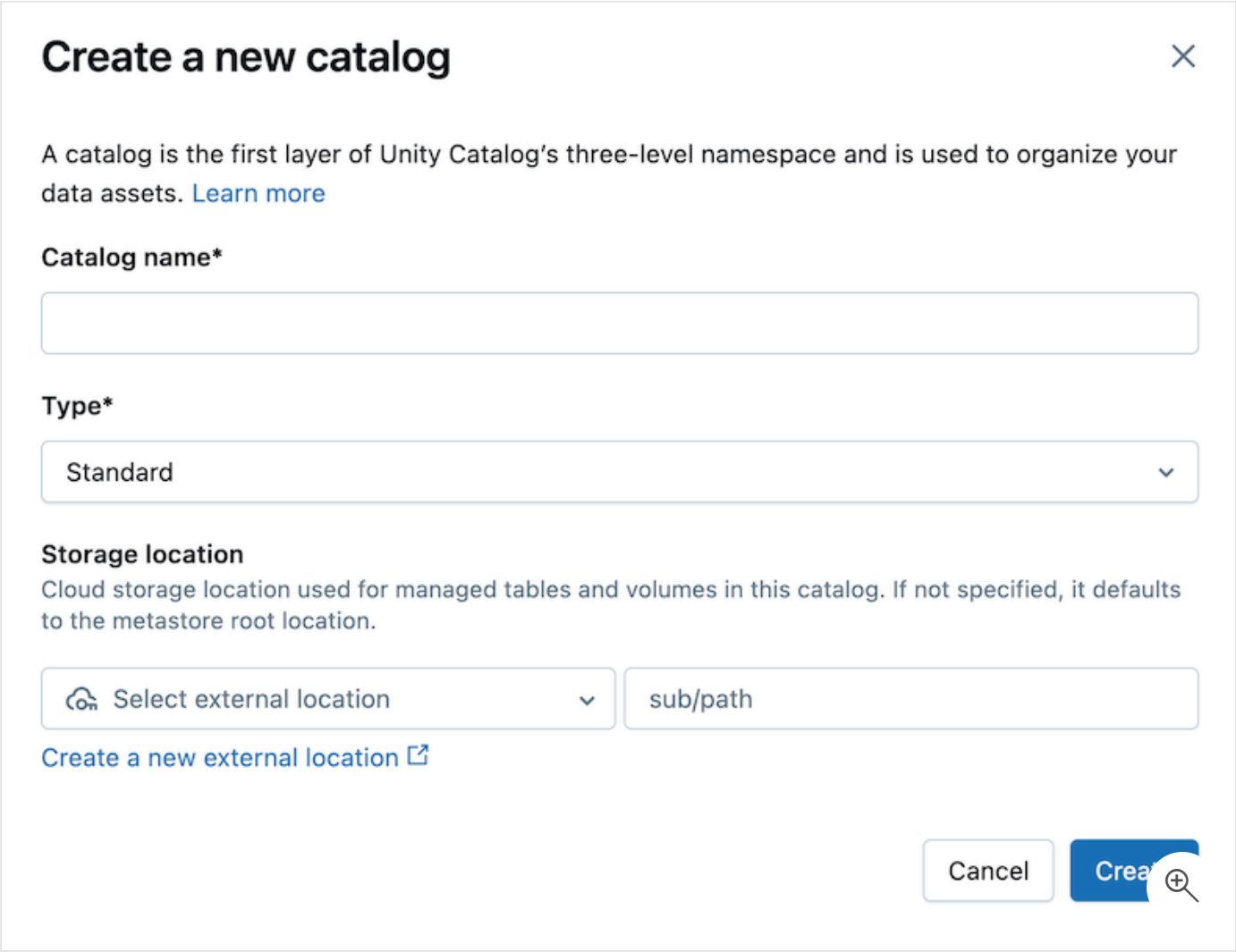
SQL

```
CREATE CATALOG IF NOT EXISTS dev_catalog
COMMENT 'Development environment for data engineering experiments';
```

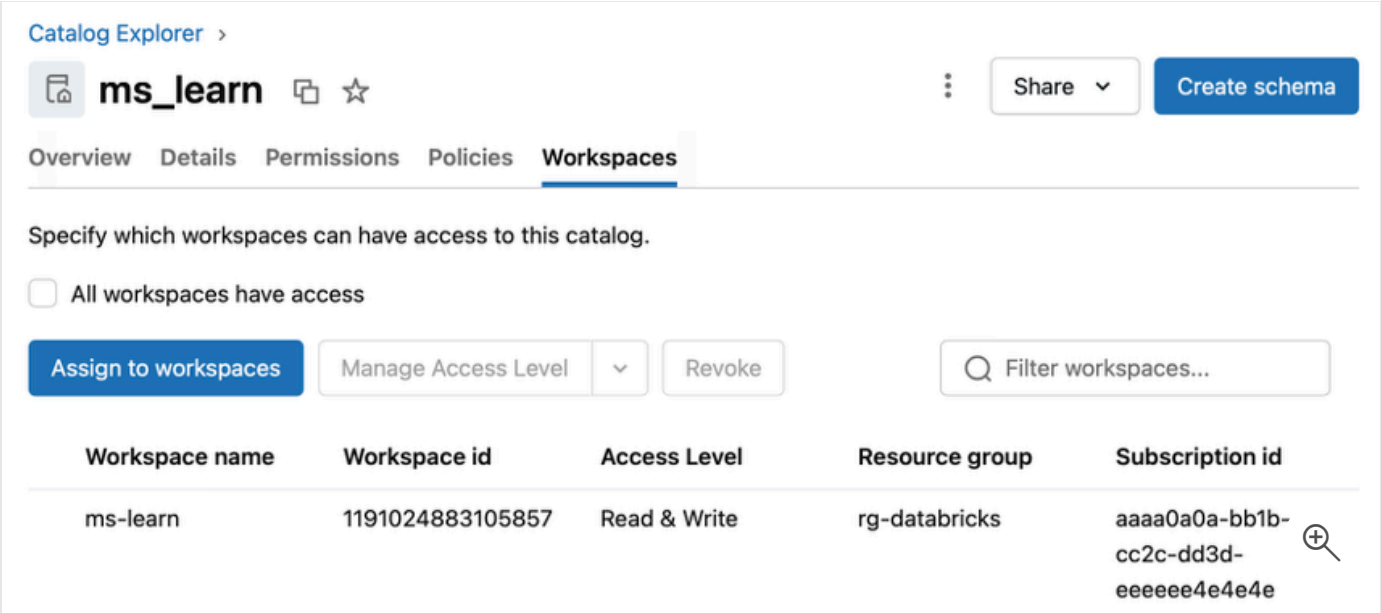
This creates a catalog named `dev_catalog` with a descriptive comment. The `IF NOT EXISTS` clause prevents errors if the catalog already exists, making your code more robust when run multiple times.

Using Catalog Explorer, you can create catalogs through the Azure Databricks UI:

1. Select **Catalog** from the left navigation.
2. Select **Create catalog**.
3. Enter a catalog name and select **Standard** as the type.
4. Optionally specify a managed storage location.
5. Select **Create**.



After creating the catalog, you configure **workspace bindings** to control which workspaces can access it. By default, new catalogs are accessible from all workspaces attached to your metastore. For production catalogs, you should restrict access to production workspaces only, ensuring that development environments can't accidentally query or modify production data.



Configure managed storage

While creating a catalog without specifying storage is possible, configuring a **managed storage location** is strongly recommended. Managed storage defines where Unity Catalog stores the data files for **managed tables** and **volumes** within your catalog. Without a catalog-level storage location, managed objects fall back to the metastore's default storage, which might not align with your organizational policies or regional requirements.

To specify managed storage, you need an **external location** already configured in Unity Catalog and the **CREATE MANAGED STORAGE privilege** on that location. The storage path must point to a cloud storage container, like an Azure Data Lake Storage Gen2 container, that meets your security and compliance requirements.

SQL

```
CREATE CATALOG IF NOT EXISTS prod_catalog
MANAGED LOCATION 'abfss://prod-data@mystorageaccount.dfs.core.windows.net/cat-
alog-root'
COMMENT 'Production catalog with dedicated ADLS Gen2 storage';
```

This command creates a catalog with dedicated production storage. All managed tables and volumes created in this catalog's schemas store their data files under this location, providing complete physical isolation from other catalogs. You can even specify a subpath within an external location, giving you flexibility to organize storage containers while maintaining centralized security credentials.

Managed storage serves two critical purposes. First, it provides **physical data isolation** by storing each catalog's data in separate cloud storage paths. When you drop a managed table, Unity Catalog can safely delete the underlying files after eight days without affecting data in other catalogs. Second, it helps you meet **compliance requirements** by ensuring sensitive data resides in specific storage accounts or regions that match your regulatory obligations.

If your metastore doesn't have a default storage location configured, you must specify managed storage when creating catalogs. This requirement ensures that Unity Catalog always knows where to place managed data files, preventing confusion and maintaining clear data governance boundaries.

Catalogs form the foundation of Unity Catalog's data organization model, providing the highest level of isolation and access control. By thoughtfully designing your catalog structure around environment boundaries or sensitivity levels, you create a governance framework that scales with your organization. Now that you understand how to create and configure catalogs, you're ready to organize the interior by adding schemas and data objects within those catalogs.

Next unit: Create schema

Previous Next